

Backpropagation

La backpropagation è l'algoritmo utilizzato per calcolare efficientemente i gradienti necessari per la discesa del gradiente.

Fasi della Backpropagation

- Forward Pass: Calcolo dell'output passando l'input attraverso la rete.
- Calcolo della Perdita: Confronto dell'output con il valore atteso tramite una funzione di costo (ad esempio, entropia incrociata o errore quadratico medio).
- Backward Pass: Applicazione della regola della catena per calcolare il gradiente della perdita rispetto a ciascun peso e bias.
- Aggiornamento dei Pesi: I pesi sono aggiornati utilizzando il gradiente calcolato e la SGD.

Fasi della Backpropagation

Forward Pass: Calcolo dell'output passando l'input attraverso la rete.

Calcolo della Perdita: Confronto dell'output con il valore atteso tramite una funzione di costo (ad esempio, entropia incrociata o errore quadratico medio).

Backward Pass: Applicazione della regola della catena per calcolare il gradiente della perdita rispetto a ciascun peso e bias.

Aggiornamento dei Pesi: I pesi sono aggiornati utilizzando il gradiente calcolato e la SGD.

Forward

$Z1 = \text{np.dot}(X_train, W1) + b1$

$A1 = \text{sigmoid}(Z1)$

$Z2 = \text{np.dot}(A1, W2) + b2$

$A2 = \text{sigmoid}(Z2)$

- X_train : matrice degli input di training, dimensione $(n_{\text{samples}}, n_{\text{input features}})$
- W_1 : pesi del primo layer (input $\rightarrow$ hidden), dimensione $(n_{\text{input features}}, n_{\text{hidden}})$
- b_1 : bias del primo layer, dimensione $(1, n_{\text{hidden}})$
- Z_1 : input lineare al primo layer
- A_1 : attivazioni del primo layer (dopo applicazione della funzione sigmoide)
- W_2 : pesi del secondo layer (hidden $\rightarrow$ output), dimensione $(n_{\text{hidden}}, n_{\text{output}})$
- b_2 : bias del secondo layer
- Z_2 : input lineare al secondo layer
- A_2 : output finale della rete, usato come predizione
- $\sigma(x)$: funzione sigmoide, definita come $\sigma(x) = \frac{1}{1+e^{-x}}$

Funzione di Loss: Cross-Entropy

$$\text{loss} = -\frac{1}{N} \sum_{i=1}^N \left[y_{\text{train}}^{(i)} \cdot \log(A_2^{(i)} + \epsilon) + (1 - y_{\text{train}}^{(i)}) \cdot \log(1 - A_2^{(i)} + \epsilon) \right]$$

- N è il numero di campioni nel dataset di training.
- $y_{\text{train}}^{(i)}$ è l'etichetta vera (target) del campione i , che può essere 0 o 1 (classificazione binaria).
- $A_2^{(i)}$ è la predizione della rete neurale per il campione i , che è il valore di output modello, compreso tra 0 e 1 grazie alla funzione sigmoide.
- ϵ è un piccolo valore (ad esempio $1e^{-8}$) aggiunto per evitare il logaritmo di zero che è indefinito.

```
loss = -np.mean(y_train * np.log(A2 + 1e-8) + (1 - y_train) * np.log(1 - A2 + 1e-8))
```

Backward Pass: Calcolo dei Gradienti

$dZ2 = A2 - y_{train}$ **#Calcolo dell'errore nel layer di output:**

$dW2 = np.dot(A1.T, dZ2) / X_{train}.shape[0]$ **#Calcolo del gradiente dei pesi (W_2) nel secondo layer**

$db2 = np.sum(dZ2, axis=0, keepdims=True) / X_{train}.shape[0] \frac{1}{N}$ **Calcolo del gradiente dei bias (b_2) nel secondo layer:**

$dZ1 = dA1 * sigmoid_derivative(Z1)$ **Calcolo dell'errore nel primo layer (hidden layer):**

$dW1 = np.dot(X_{train}.T, dZ1) / X_{train}.shape[0]$ **Calcolo del gradiente dei pesi (W_1) nel primo layer:**

$db1 = np.sum(dZ1, axis=0, keepdims=True) / X_{train}.shape[0]$ **Calcolo del gradiente dei bias (b_1) nel primo layer:**

Similmente a $db2$, il gradiente per i bias è la somma dell'errore $dZ1$ nel primo layer, media con il numero di campioni.

- Questo valore verrà utilizzato per aggiornare i bias del primo layer.
- $W1 -= learning_rate * dW1$
- $W2 -= learning_rate * dW2$
- $b2 -= learning_rate * db2$

Cosa fare per il multiclass?

- Per modificare la rete neurale da una versione binary classification a una multiclass classification, sono state apportate alcune modifiche chiave nel codice.
- Queste modifiche riguardano principalmente la funzione di attivazione nell'output layer, la funzione di perdita (loss function), e la gestione dei gradienti.

Funzione di attivazione per il layer di output: Softmax

Data una lista di valori reali (detti *logits*):

$$\mathbf{z} = (z_1, z_2, \dots, z_K)$$

la funzione **softmax** restituisce una lista di valori:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Funzione di perdita: Categorical Cross-Entropy

$$L = - \sum_{i=1}^K y_i \log(p_i)$$

Derivata della funzione di perdita (per backpropagation) ¶

- Derivata della funzione di perdita (per backpropagation)
- Modifica: La derivata della funzione di perdita è stata adattata per il caso multiclass. In particolare, la derivata della categorical cross-entropy combinata con softmax è più semplice rispetto a un modello binario e può essere espressa come la differenza tra la probabilità predetta e l'etichetta vera (one-hot encoded).
- Motivo: La derivata combinata tra softmax e categorical cross-entropy è diretta e semplifica il calcolo durante il backward pass.

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{1}{N} (y_{\text{pred}} - y_{\text{true}})$$